Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 04

Lecture – 21

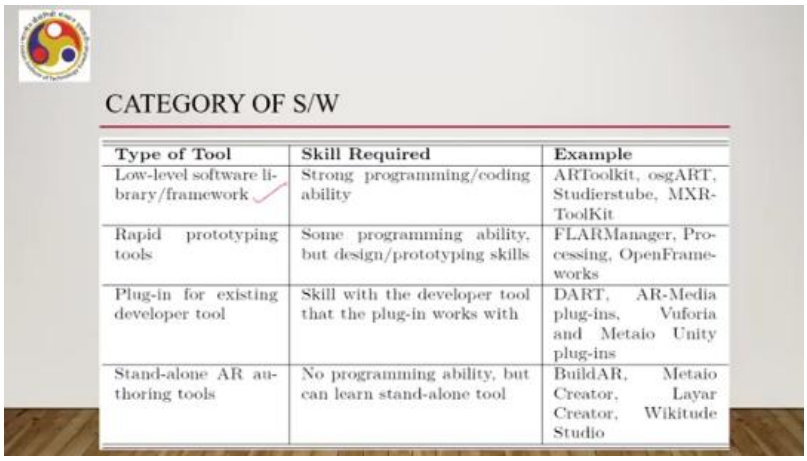
Lec-21: AR Software

Hello and welcome to the NPTEL MOOCs course on Foundations for Virtual and Augmented Reality Systems. We are going to start a new lecture, Lecture Number 11, on AR or Augmented Reality Software. So, just a quick recap. So, in the previous lecture, we talked about virtual reality software. There we talked about various components that are desirable in a virtual reality software.

with respect to one abstraction that we called virtual world generator, and also we have gone through some popular tools and packages and software, that are available for different hardware platforms or for building and deploying systems on the web. So, in this lecture, we are going to do the same for AR or augmented reality, the other technology area that that we are discussing. Like VR, in the case of augmented reality also, there are numerous software tools for implementation.

So, a developer can access a large number of such tools. Now, the thing is, for a better understanding, a structured view of those tools will be beneficial So, what we will do is we will try to arrange all the tools that are available in the form of a hierarchy depending on decreasing programming skills required. So, some tools may require a very high level of skill, whereas others may require no skills at all; accordingly, we will try to organize them in some categories. So, at one extreme of the category are the low-level software libraries that require considerable skill on the part of the developer. Skill in terms of understanding the library functions as well as skills in terms of coding where the library functions can be called. So, that is one extreme where low-level libraries are used. At the other extreme, we have simple authoring tools which assume that the developer does not have any programming skill.

So, the tools are typically for novice developers or novice users; users themselves can be developers which assumes that no programming skill is required. Now, if we categorize it in this way, then we can come up with four categories: low-level software libraries or frameworks, rapid prototyping tools, plugins for existing developer tools, and finally, standalone AR authoring tools. So, this is one extreme where no coding skill may be required, whereas this other one, low-level libraries, is another extreme where a significant amount of coding skills and programming skills from the developer's side will be required.



CATEGORY OF S/W

Type of Tool	Skill Required	Example
Low-level software library/framework	Strong programming/coding ability	ARToolkit, osgART, Studierstube, MXR-ToolKit
Rapid prototyping tools	Some programming ability, but design/prototyping skills	FLARManager, Processing, OpenFrameworks
Plug-in for existing developer tool	Skill with the developer tool that the plug-in works with	DART, AR-Media plug-ins, Vuforia and Metaio Unity plug-ins
Stand-alone AR authoring tools	No programming ability, but can learn stand-alone tool	BuildAR, Metaio Creator, Layar Creator, Wikitude Studio

So, skill-wise, the low-level software libraries or frameworks require strong programming or coding ability. Some examples of such libraries or frameworks are AR Toolkit, OSG Art, StudiersTube, MXR Toolkit, and so on. Some of these are no longer available, but those are there. In the second category, rapid prototyping tools, the skills required are some programming ability, lower level as compared to the previous one, but design prototyping skills may be required for using such tools. Examples of such tools are FLAR manager, OpenFrameworks, and so on. Then we have plugins for existing developer tools. Here the skill required will be the skill with the original tool with which the plugin is being used.

So, the developer needs to know the original tool where the plugin is being used. And finally, standalone AR authoring tools. These do not assume any programming ability So, without any knowledge, such tools can be used. So, examples for plugins are AR media plugins, Vuforia, MetaIO Unity plugins, Dart, and so on.

Some of these were very popular earlier, but are no longer being supported. Some are quite popular still today, such as Vuforia. Standalone AR tools examples are BuildAR, MetaIO Creator, Wikitude, and so on. Again, some of these are no longer supported. So, one way of categorization is based on the skill.

There can be other types of categorizations also. Sometimes we can categorize based on target application platforms. So, some tools are meant for desktop AR applications while others are meant for mobile AR applications. Again, categorization can be based on the technology. So, some support CV-based tracking, computer vision-based tracking, whereas others use other sensors for tracking such as GPS sensors.

So, such technology-based categorization or platform-based categorization or skill-based categorization is possible with AR tools. But we should try to understand the tools in terms of some sort of categorization for a structured view. So let us assume the skill-based categorization and accordingly let us discuss the tools. So we'll start with the the one which requires the maximum skill from the developer, that is the low-level libraries and frameworks. So, what is the idea? Low-level libraries provide access to the core tracking and display functionality to build an AR application. A popular example is AR Toolkit. There are many other such examples.

Now, if we talk about AR Toolkit. It is one of the most widely adopted software libraries for developing AR applications. It can provide marker-based tracking in its open-source version, or there is a commercial version available for AR toolkit which can be accessed from AR tool works, which also supports tracking natural features on an arbitrary image, that is, it uses computer vision techniques. The toolkit is written in C, and it supports various platforms such as Windows, Linux, and macOS.

One thing to note here is that the toolkit, ARToolKit, is primarily meant to provide tracking support. Tracking we have already discussed in one of the previous lectures. It tracks various things including objects in the environment, and AR and VR systems are supposed to have tracking functions inbuilt. Now, if we are using ARToolKit to develop an AR application, then the toolkit is a library which provides functions that can enable the developer to perform tracking and accordingly do some activity. The primary purpose of ARToolKit is to provide tracking support to the developer.

As a consequence, the toolkit provides only low-level OpenGL-based rendering support. So, very primitive rendering support is provided and no built-in support for interaction techniques or complex realistic graphics. So, it can be used to render, but at a very low level, very simple level rather than realistic graphics, and also it does not provide support for interaction with the digital content overlaid on the real-world view. So, if we are going to use it alone, it may still result in some system, but that system will be very rudimentary, and to provide interaction the developer has to write things from scratch, develop things

from scratch. So, it will be difficult and time-consuming to develop a complete AR application which looks realistic. Now, that is the problem if we are going to use the ARToolKit alone. So, to overcome this problem, what we can do is. Now, this problem typically happens with many standard toolkits or low-level libraries.

They primarily provide tracking support and miss certain desired features that are required for realistic rendering of digital content. on real-world view that is typically required in an AR application, and also provide interaction support. So to overcome that, what we can do is we can combine multiple libraries and create a framework for AR system development. So we can call the combination of those libraries a framework for the developers. Let us see an example with AR toolkit, what kind of framework we can use or create.

So, we can overcome the limitations of AR toolkit by combining other libraries. Typically, we should look for libraries that provide interaction as well as realistic graphic support. One such type of library can be libraries with support for scene graph management. Now, why are we emphasizing the term scene graphs? Scene graphs represent computer graphic scenes as a node-based graph.

So, essentially the entire scene is represented in the form of a tree with nodes and links between nodes, and we can selectively work with nodes to manipulate the scene and also create realistic effects and also allow interaction. So it is actually a very nice way of trying to represent the complexity of a scene as well as trying to work with the complexity of a scene. So it provides a way to manage the functionality of a complex graphics application. So scene graph-based representations generally aid in graphics applications.

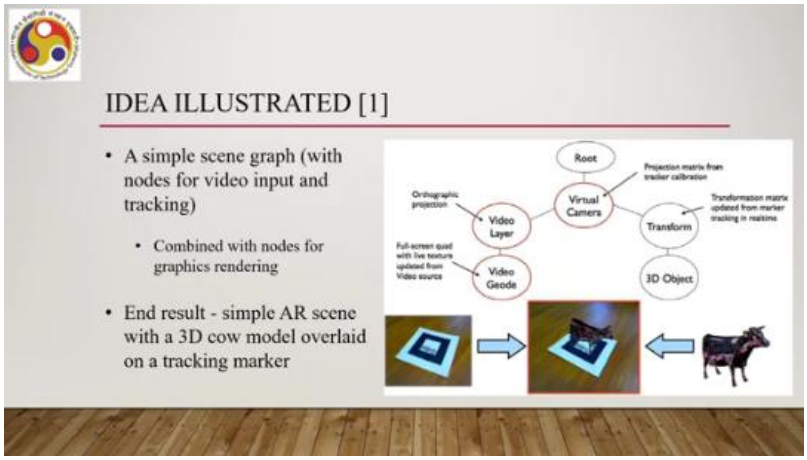
Some of the features that typically scene graph-based representations provide have. So, they allow sorting objects according to visibility: which are visible with respect to viewer position and which should not be visible. It can allow removing objects out of camera field of view. So, this one is called visible surface detection, and this one is called clipping.

So, these operations that are typically part of any 3D graphics pipeline are easily managed and supported by this type of representation. Also, it can allow state management and higher level abstractions of functionalities. So, many things are possible if we go with scene graph-based representation. So, in a sense, if we summarize if we take help of a scene graph library, then that can enable the developer to focus on interactions between virtual objects rather than low-level object rendering. So, earlier with only AR Toolkit, the focus was on tracking. So, low-level object rendering plus interaction both had to be taken care of by the developer. Now, if we combine the AR toolkit with scene graph libraries, Any of the scene

graph libraries, then these low level rendering challenges can be taken care of by the library and the developer can focus more on the interaction part.

So, one such open source library, one such scene graph library is The Open Scene Graph, it used to be quite popular earlier. Now, there is a successor of OSG that is Vulkan Scene Graph VSG. Both are open source and widely used. There are other such libraries as well.

We are taking these as examples. Now, it is written in C++ language. And such libraries are built on top of another library that is called OpenGL, which accesses the lower-level hardware. The advantage is that with this library is that it supports a large number of plugins for additional functionality such as model loading, shadow rendering, physics which are primarily relevant most of the time for VR and partly relevant for AR. Through this example, we can try to understand the concept of a framework based on AR toolkit and OSG. So, it is, suppose we have this marker on a table; when we point our mobile phone, the phone detects the marker and then a 3D object, that is a cow, pops up on your screen. So, then it involves two parts: tracking, that is recognizing the marker and its position, and then 3D registration, that is creating and rendering the cow on that position.



So, these operations can be taken care of by combining AR toolkit and OSG or VSG, its successor. The entire thing can be represented in the form of a node graph. So, we can have a root node, then the virtual camera can be represented as a node and we can work with parameters of the node. Projection that is part of rendering can be considered as another node which allows projection on the video layer.

Other related concepts can be considered as nodes. The 3D object itself, the cow, can be considered as a node. Then various transformations that are part of 3D graphics can be represented as a single node or individual nodes. So, once we are able to represent this

entire activity in the form of a graph, then it is easier to visualize and manipulate such a graph to create the desired effect that the user is looking for.

So, here the simple scene graph contains video input and tracking. Both video input primarily comes from the input device. Tracking again based on sensors present in the input device, in this case camera, combined with nodes for graphics rendering. So, various nodes like transformation, 3D object representation, orthographic projection; these are related to graphics rendering. So, when we combine these, the end result is a simple AR scene with a 3D cow model overlaid on a tracking marker. Now, this part overlaid on a tracking marker that is 3D registration and tracking is taken care of by AR Toolkit whereas the other part, AR scene creation with a 3D cow that is taken care of by the OSG that is Open Scene Graph. So, there are many such frameworks available, some are still in use, some were being used earlier quite popular for example OSG ART 3 library it links AR Toolkit for tracking support with OSG library for real-time rendering features Goblin XNA combines ART tag tracking library so this ART tag is another tracking library with Microsoft XNA game platform Metaio SDK plus OSG the former provides a comprehensive set of AR tracking technologies Metaio SDK whereas OSG provides rendering support. So this is one way of looking at the software. Now there can be mobile-specific frameworks as well.

For example, AR Toolkit for mobile. It is a version of the AR Toolkit available for mobile platforms including iOS and Android, integrated with OpenSceneGraph but developed for mobile. Then we have DroidAR. And also we have Vuforia. Vuforia is still quite popularly used.

Now, various libraries, as we have mentioned earlier, support tracking, but they can support it in different ways. For example, computer vision can be used to support tracking, which is there in Vuforia, or other sensors can be used for tracking support. For example, DroidAR uses other sensors for tracking support. That is about low-level libraries and frameworks. Next, let us shift our discussion to the second category: rapid prototyping and development tools.

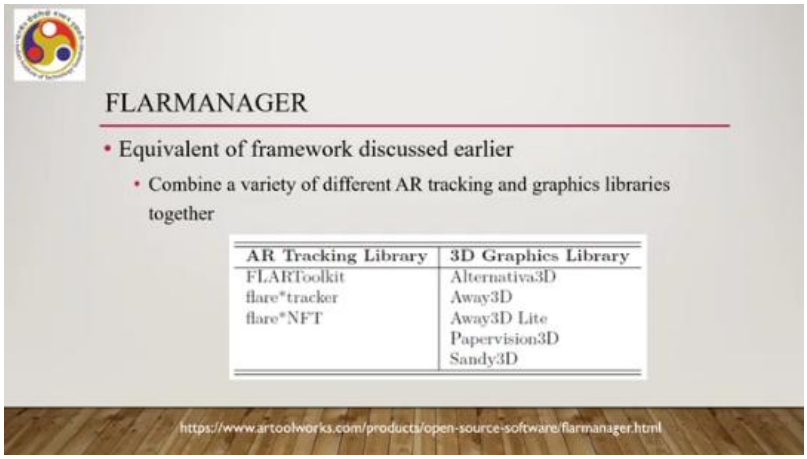
Now, these low-level libraries and frameworks provide maximum flexibility in development. We all know that if you have a skill and you understand the libraries, you can combine them in any way you want and can come up with your own design. But it can be time consuming and requires significant programming skill. To implement an AR application using the library, first you have to know the libraries or the framework and then write programs to implement the tool. So that requires time and skill.

Sometimes it is helpful to rapidly prototype or visualize an idea to show to the end users or clients. We all know the importance of prototypes. Before actually undertaking any major development exercise that involves coding. So, if some idea is not good, then no further coding will be required. So, if we can visualize such ideas beforehand using simple means rather than actually going through the entire coding exercise, then that will be beneficial as we all understand.

So one such popular authoring tool for creating prototypes having multimedia content for the web and desktop platform is provided by the popular Adobe Flash tool. So that tool can be used to prototype a variety of desktop and mobile experiences by using experiences related to AR by using AR plugins that are available such as FLAR Toolkit or FLAR Manager. Now, let us look at the FLAR Toolkit. It is a version of AR toolkit ported over to Flash.

It allows people to have AR experience from within their web browser. If a user is using the toolkit, the FLAR toolkit, to prototype an AR system or AR idea, then with a few lines of code, the user can activate their camera to get the experience, of course. look for AR markers in the camera view; tracking that is part of tracking. load and overlay virtual content onto the tracked image for 3D registering.

So, all three activities, or the two primary activities like tracking and 3D can be implemented using the FLAR Toolkit or FLAR toolkit. In that way, since all of these can be done with a few lines of code, then it makes it possible for anyone with a FLASH-enabled web browser and camera connected to a computer to have or develop an AR experience with minimal effort. Similarly, there is a framework called the Flash AR Manager or FLAR Manager.



It is the equivalent of a framework we discussed earlier. It combines a variety of different AR tracking and graphics libraries together, but primarily for prototyping purposes. So, combinations can be in different ways. So, if on the one hand we have an AR tracking library, we can use the FLAR Toolkit. FLAR Tracker, NFT.

And on the other hand, if we are combining it with a 3D graphics library, then we can use alternative 3D, Away 3D, Away 3D Lite, Paper Vision 3D, Sandy 3D. So, these are some examples of different graphics libraries that can be combined with the toolkits. But both in this case can be used; a combination of both can be used. For primarily for prototyping rather than going for a full-fledged development exercise. Although we have discussed such prototyping tools using Flash only, you can understand that Flash-based tools are not the only ones available.

Non-Flash-based authoring tools are also there, but they may not be as popular as the Flash-based tools because of the penetration in the market. Now, such other tools we are going to discuss in the next part of the lecture. So, just to summarize whatever we have covered up to this point. We have got the broader idea of AR software, particularly in terms of a structured representation using categories.

So, different ways to categorize them we have seen: either skill-based categorization, skill requirement-based categorization, or platform-based categorization, or technology-based categorization. We are following skill requirement-based categorization, and based on that we have started our discussion with the low-level libraries and frameworks, then we moved to authoring tools for building prototypes, which we are continuing. So, we have covered that category partly. We are going to finish that category in the next lecture along with other categories.

I hope you have enjoyed and understood the concepts. See you in the next lecture. Goodbye. Thank you, and happy learning.